Unification in Lean4

Maximilian Keßler, April 21, 2026

1 Usage of unification in Lean

In Lean, unification takes place whenever the elaborator needs to unify two "partially typed" terms that contain metavariables. The goal is to infer a suitable substitution for the metavariables in question such that resulting terms are *definitionally equal* (at the current transparency level). This typically happens whenever

- We leave arguments { in brackets } implicit and want Lean to deduce them:
  In `example (xs ys : List ℕ) : List.append xs ys`, Lean infers the implicit first argument ℕ of `List.append`.
- We left out type annotations to be inferred, e.g. at quantifiers, context declarations or types of expressions that we are defining.
- We need to fill holes in terms that were left unspecified by writing `_`.
- We leave motives $\alpha \to$ `Prop` implicit when using `induction` or theorems like `Eq.subst`.
  In `example (a b : ℕ) (R : ℕ → ℕ → Prop) (e : a = b) (H : R a a) : R a b := Eq.subst e H`, Lean needs to infer the motive `m := fun n ↦ R a n` such that `m a = H` and `m b = R a b`.

2 Mathematical Theory: Unification in simply-typed λ-calculus

Unification is the question of finding a *substitution* to make a (set of) equations between terms *equal*. Here, *equality* means that terms have the same (long) normal form after $\beta\eta$-reduction up to α-conversion. We distinguish two cases:

First-Order unification: There are efficient algorithms to solve any First-Order-unification problem in almost linear time. The core ideas are:

- *Term reduction*: For function symbols f, $f\ x_1\ \ldots = f\ y_1\ \ldots$ already yields the requirements $x_1 = y_1, \ldots$, transforming into an equivalent, simpler problem.
- *Variable elimination*: If we know $x = t$ for a variable x and a term t, we can substitute t for x everywhere to eliminate x.

Higher-Order unification: Starting with Second-Order unification, the unification problem becomes undecidable, hence we cannot hope for a general-case algorithm. A basic proof idea (for Third-Order unification) is to reduce from Hilbert's tenth problem (determining whether multivariate polynomial equations over ℕ have solutions is undecidable) by encoding polynomials and their evaluation into λ-calculus using *Church natural numbers*.

3 Unification in Lean

Analogous to $\beta\eta$-reduction, in Lean we use *definitional equality* to formulate the unification problem. Many of the occurring unification problems in practice are actually First-Order. Lean's elaborator implements First-Order unification, while also handling the following additional complexities simultaneously:

- Dependent types and type universes
- Inserting *coercions* to "convert" between types
- Respecting *transparency levels* regarding unfolding behavior of definitions
- Typeclass synthesis

Additionally, Lean uses Higher-Order *pattern unification* to solve a subclass of Higher-Order unification problems. When X is a metavariable of Higher-Order (i.e. a function type), X can be inferred as a λ-abstraction whenever X is applied to distinct bound variables. This mechanism powers tactics like `induction`.

For example, in the equality $\lambda x\ \lambda y\ (X\ x\ y) = \lambda x\ \lambda y\ (x\ (\mathrm{suc}\, y))$, we can infer $X = \lambda x\ \lambda y\ x\ (\mathrm{suc}\, y)$ to unify the two sides terms.

References

[AP18] Andreas Abel and Brigitte Pientka. *Extensions to Miller's Pattern Unification for Dependent Types and Records*. https://www.cs.mcgill.ca/~bpientka/papers/unif_miller60.pdf. Technical manuscript (online version). 2018.

[Dow01] Gilles Dowek. "Higher-Order Unification and Matching". In: *Handbook of Automated Reasoning*. Ed. by Alan Robinson and Andrei Voronkov. Amsterdam: North-Holland, 2001, pp. 1009–1062. DOI: 10.1016/B978-044450813-3/50018-7. URL: https://web.archive.org/web/20250722104018/https://lsv.ens-paris-saclay.fr/~dowek/unification.pdf.

[Gol81] Warren D. Goldfarb. "The undecidability of the second-order unification problem". In: *Theoretical Computer Science* 13.2 (1981), pp. 225–230. DOI: 10.1016/0304-3975(81)90040-2. URL: https://www.sciencedirect.com/science/article/pii/0304397581900402.

[Hue73] Gérard P. Huet. "The undecidability of unification in third order logic". In: *Information and Control* 22.3 (1973), pp. 257–267. DOI: 10.1016/S0019-9958(73)90301-X. URL: https://www.sciencedirect.com/science/article/pii/S001999587390301X.

[Hue75] Gérard P. Huet. "A unification algorithm for typed lambda-calculus". In: *Theoretical Computer Science* 1.1 (1975), pp. 27–57. DOI: 10.1016/0304-3975(75)90011-0. URL: https://www.sciencedirect.com/science/article/pii/0304397575900110.

[Lea] Lean Prover Community. *Elaboration*. Lean 4 Metaprogramming Book, accessed 2026-04-21. URL: https://leanprover-community.github.io/lean4-metaprogramming-book/main/07_elaboration.html.

[Mil91] Dale Miller. "Unification of Simply Typed Lambda-Terms as Logic Programming". In: *Proceedings of the Eighth International Conference on Logic Programming*. MIT Press, 1991, pp. 255–269. URL: https://www.lix.polytechnique.fr/Labo/Dale.Miller/papers/iclp91.pdf.

[MM82] Alberto Martelli and Ugo Montanari. "An Efficient Unification Algorithm". In: *ACM Transactions on Programming Languages and Systems* 4.2 (1982), pp. 258–282. DOI: 10.1145/357162.357169.

[Mou+15a] Leonardo de Moura, Jeremy Avigad, Soonho Kong, and Cody Roux. *Elaboration in Dependent Type Theory*. 2015. arXiv: 1505.04324 [cs.LO]. URL: https://arxiv.org/abs/1505.04324.

[Mou+15b] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. "The Lean Theorem Prover (System Description)". In: *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*. 2015, pp. 378–388. DOI: 10.1007/978-3-319-21401-6_26. URL: https://doi.org/10.1007/978-3-319-21401-6_26.